

Chapter 2: File and Stream I/O

How to read these notes

- **MUST KNOW** Sections 2.1 to 2.4. Required to know well, since every project reads data from files.
- **TAKEAWAY ONLY** Sections 2.5 and 2.6. These are I/O speed optimizations. Only the boxed takeaway really matters, the rest is explanation of why it works.
- **NOT REQUIRED** Sections 2.7 to 2.9, plus 2.4.3. Useful to know, but you will not be required to use any of it in this class. Boxed with a dashed red border.

2.1 Standard I/O Channels **MUST KNOW**

2.1.1 The I/O Stream Library

- A **stream** is an abstraction that lets programs send and receive data sequentially.
- **Extraction operator** `operator>>`: pulls data out of a stream.
- **Insertion operator** `operator<<`: puts data into a stream.
- The `<iostream>` library implements most stream functionality. It is object-oriented, so streams are objects arranged in an inheritance hierarchy:

```
std::ios_base
|
+-- std::ios
|   |
|   +-- std::istream          (input)
|   |   +-- std::ifstream
|   |   +-- std::istringstream
|   +-- std::ostream          (output)
|   |   +-- std::ofstream
|   |   +-- std::ostringstream
|   +-- std::iostream          (inherits from BOTH istream and ostream)
|       +-- std::fstream
|       +-- std::stringstream
```

- `std::ios_base` is the base class of the whole library.
- `std::iostream` inherits from **both** `std::istream` (input) and `std::ostream` (output), so it can do both input and output.
- `<ios>`, `<istream>`, and `<ostream>` are not usually included explicitly, because derived class headers like `<iostream>` include them automatically.

2.1.2 Cin, Cout, and Cerr

- **`std::cin`**: an object of type `std::istream`, the **standard input stream** for character input (the name is short for character input). It accepts input from a standard input device such as a keyboard. Read from it with `operator>>`.

```
std::string name;
std::cin >> name;
```

- Extraction **fails** if the input cannot be interpreted as the type being read into:

```
int32_t val;
std::cin >> val; // the next item in the stream must be an int
```

- **`std::cout`**: an object of type `std::ostream`, the **standard output stream** for character output. It displays characters on a standard output device such as your console. Write to it with `operator<<`.

```
std::string text = "This text is printed out as program output.";
std::cout << text;
```

- **`std::cerr`**: an object of type `std::ostream` representing the **standard error stream**. It is distinct from the program output stream, which lets you work with normal output and error output separately. Also uses `operator<<`.

```
std::string text = "This text is printed out as error message output.";
std::cerr << text;
```

- Insertion and extraction can be **chained** in one expression:

```
std::string str = "EECS";
int32_t num = 281;
std::cout << str << num; // prints "EECS281"
```

Remark: printing booleans

Stream operations treat bools as the integers 0 and 1, not as text.

```
bool b = true;
std::cout << b; // prints 1
std::cout << std::boolalpha << b; // prints true
```

The same keyword works on input, letting you read the words "true" and "false" into a bool:

```
bool b;
std::cin >> std::boolalpha >> b; // user inputs the word "true"
// b is true (would not work without boolalpha)
```

2.2 Input and Output Redirection **MUST KNOW**

- **Input redirection** sends the contents of a file into the `std::cin` standard input stream. Your program then reads the file by extracting from `std::cin`, treating the file contents as if a user typed them.
- Running a program normally makes the user type input by hand:

```
./myProgram
```

```
myProgram.cpp  ->  compile and run  ->  user types into console
(reads item and  ./myProgram        Apple 0.99
price using cin)                               Banana 0.89 ...
```

- With all of the input already in a file (for example `fruits.txt` holding lines like `Apple 0.99`), redirect it instead:

```
./myProgram < fruits.txt
```

- `<` is the **input redirection operator**. It redirects the contents of `fruits.txt` into `std::cin` so the program can extract them with operator `>>`.
- **Output redirection** works the other way. `>` sends the contents of `std::cout` to a file:

```
./myProgram > report.txt
```

- Both can be combined in a single command:

```
./myProgram < fruits.txt > report.txt
```

- With redirection, everything extracted from `std::cin` comes from the file after `<`, and everything printed to `std::cout` goes to the file after `>`.

Remark: error redirection

Redirection also works on `std::cerr` using the **error redirection operator** `2>`:

```
./myProgram 2> error_report.txt
```

2.3 Reading Input in a Loop MUST KNOW

- `std::cin` and other `std::istream` objects can be **implicitly converted to a bool**.
 - No errors and ready for extraction: converts to `true`.
 - In an error state: converts to `false`.
- Because of this, you can read an entire file by extracting from `std::cin` in the condition of a while loop. The loop ends automatically once there is nothing left to read:

```
std::string word;
while (std::cin >> word) {
    // word stores the next word in the input stream
} // while
```

- Reading a stream inside a while loop like this is the best way to guarantee everything is read correctly, especially when input is redirected from a file rather than a keyboard.

Do not use `cin.good()`, `cin.eof()`, `cin.bad()`, or `cin.fail()` as the condition of a while loop.

These functions do not return false until **after** an operation has already failed, so the loop runs one extra time on invalid input.

Example with a file of 100 values: after reading the 100th value, the program starts iteration 101 and checks if `std::cin` is still good. There is no 101st value, but nothing has failed yet, so the loop continues. Only when the program tries to read the 101st value does it discover the value does not exist and `cin.good()` becomes false. By then it is too late, the read has already failed.

2.4 Stream Extraction MUST KNOW

2.4.1 The Extraction Operator

- operator `>>` behavior:
 - **Ignores leading whitespace.**
 - Consumes as many items as possible for the type being read into, until it hits whitespace or the **end-of-file flag** (a special flag meaning there is no more data).
 - Stops reading if the stream is in an **error state**, which happens when the next element does not match the type being read into.
 - Returns a reference to the `std::istream` it read from, which can be implicitly converted to a bool.

Example file (`.` is a space, `\n` is a newline):

```
input.txt
...here...is...a\n
...file...to...be\n
\n
read
```

```
std::string word;
while (std::cin >> word) {
    std::cout << word;
} // while
```

- The operator skips the whitespace before “here”, then consumes each character until the next whitespace. Repeating this for the whole file

gives:

```
output.txt
hereisafiletoberead
```

- Extraction **removes** what it reads from the stream and always continues where you left off. You can read the first five values, do other work, then come back and read the sixth. You never restart from the beginning of the file.

2.4.2 Stream Getline

```
std::istream& std::getline(std::istream& is, std::string& str, char delim);
```

- Extracts characters from input stream `is` into string `str` until the delimiting character `delim` is found. If `delim` is not given, the newline `'\n'` is used.

```
std::istream& std::getline(std::istream& is, std::string& str);
```

- Parameters: (1) the input stream to read from, (2) the destination string, (3) the optional delimiting character.
- `std::getline()` consumes **all** characters, including whitespace, up to the delimiter. The delimiter itself is **discarded**, and everything before it is stored.
- Like the extraction operator, it returns a reference to the input stream, so it can be used as a while loop condition.

```
input.txt
...here...is...a
...file...to...be
read
```

```
std::string line;
int32_t counter = 1;
while (std::getline(std::cin, line)) {
    std::cout << "line " << counter++ << ": " << line << std::endl;
} // while
```

```
output.txt
line 1: ...here...is...a
line 2: ...file...to...be
line 3:
line 4: read
```

- Spaces are kept, only the delimiter is dropped. On the third iteration nothing sits between the two newlines, so line 3 is empty.

Custom delimiter

Input file with no newlines:

```
input.txt
0|12332|99:74:11:21:61|TCP|Connection attempt failed|149|36240|...|OS|System failure
```

```
std::string line;
int32_t counter = 1;
char delim = '|';
while (std::getline(std::cin, line, delim)) {
    std::cout << "line " << counter++ << ": " << line << std::endl;
} // while
```

```
line 1: 0
line 2: 12332
line 3: 99:74:11:21:61
line 4: TCP
line 5: Connection attempt failed
... (one line per field, up to)
line 14: OS
line 15: System failure
```

Common mistake: mixing >> and getline on the same line of input

Both start where you last left off, so they can end up fighting over the same line.

```
input.txt
2
apple
banana
```

```
int32_t num_fruits;
std::cin >> num_fruits;
std::string line;
for (int32_t i = 1; i <= num_fruits; ++i) {
    std::getline(std::cin, line);
    std::cout << "fruit " << i << ": " << line << std::endl;
} // for i
```

Actual output:

```
fruit 1:
fruit 2: apple
```

- After `>>` extracts the `2`, the next character still in the stream is the newline. The first `std::getline()` reads up to that newline and gets **nothing**, so "banana" is never reached.
- Fix 1:** an extra `std::getline()` call to clear the leftover input:

```
std::cin >> num_fruits;
std::getline(std::cin, line); // clear out everything before the next line
for (int32_t i = 1; i <= num_fruits; ++i) {
    std::getline(std::cin, line);
    std::cout << "fruit " << i << ": " << line << std::endl;
} // for i
```

- **Fix 2 (cleaner):** use `std::istream::ignore()` whenever you switch from `>>` to `getline()`. Include `<limits>` to use `std::numeric_limits<>`.

```
std::cin >> num_fruits;
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // remove whitespace and newline
for (int32_t i = 1; i <= num_fruits; ++i) {
    std::getline(std::cin, line);
    std::cout << "fruit " << i << ": " << line << std::endl;
} // for i
```

2.4.3 Stream Ignore

Marked as extra material in the notes. NOT REQUIRED

```
std::istream& std::istream::ignore(std::streamsize n, int delim);
```

Extracts and discards, whichever comes first:

- a. `n` characters from the input stream, or
- b. everything up to **and including** `delim`, if `delim` is encountered, or
- c. everything up to the end of the file or stream.

Defaults: `n` is 1 if not specified; `delim` is a special end-of-file identifier if not specified. If you do not care about `n` and just want to discard up to a delimiter, set `n` to `std::numeric_limits<std::streamsize>::max()`.

Input file with a label before the number:

```
input.txt
Count: 2
apple
banana
```

Approach with a junk variable:

```
std::string junk;
std::cin >> junk; // reads in "Count:"
int32_t num_fruits;
std::cin >> num_fruits; // reads in 2
std::string line;
for (int32_t i = 1; i <= num_fruits; ++i) {
    std::getline(std::cin, line);
    if (line.find_first_not_of(" ") == std::string::npos) { // skip all-whitespace lines
        --i;
        continue;
    } // if
    std::cout << "fruit " << i << ": " << line << std::endl;
} // for i
```

Cleaner approach with `ignore()`:

```
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), ' '); // up to and including first space
int32_t num_fruits;
std::cin >> num_fruits; // reads in 2
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // up to the next newline
std::string line;
for (int32_t i = 1; i <= num_fruits; ++i) {
    std::getline(std::cin, line);
    std::cout << "fruit " << i << ": " << line << std::endl;
} // for i
```

Ignoring a fixed number of characters instead of using a delimiter, with `input.txt` holding `123456789`:

```
std::cin.ignore(5); // extract and discard the first five characters
int32_t num;
std::cin >> num;
std::cout << num << std::endl; // prints 6789
```

2.4.4 Summary of Stream Operations

The two most important extraction methods, on the file `abc.defghi`:

```
operator>>
Extracts as many items as possible for the type being read into,
until whitespace/newline or end-of-file.

input stream: a b c . d e f g h i
std::string str;
std::cin >> str; -> str = "abc"
input stream: . d e f g h i (whitespace ignored and discarded)
```

```
std::getline()
Extracts as many items as possible into a string, until a newline
(or specified delimiter) or end-of-file.

input stream: a b c . d e f g h i
std::string str;
std::getline(std::cin, str); -> str = "abc def"
input stream: g h i          (newline ignored and discarded)
```

2.5 Stream Buffers and Flushing TAKEAWAY ONLY

The takeaway: when printing to `std::cout`, it is faster to print a newline with `'\n'` than with `std::endl`.

- Data sent to an output stream is not always written to the console (or file) immediately. It may be **buffered** first.
- A **buffer** is a block of memory allocated to a stream object to hold temporary stream data. Writing in blocks is much faster than writing one character at a time.
- A buffer is normally emptied (**flushed**) when it fills up. You can flush manually with the `std::flush` manipulator:

```
std::cout << "Hello World"; // sent to the cout output buffer
std::cout << std::flush;    // flushes the buffer, text written to console
```

- `std::cout` is **block buffered**: characters are stored and written in chunks. `std::cin` also buffers input so values are read and processed in chunks.
- Exception: `std::cerr` is **unbuffered** by default, so anything sent to it appears at its destination immediately.

Why endl is slower

```
for (int32_t i = 0; i < 100000; ++i) {
    std::cout << "Hello World" << std::endl; // slower
} // for i

for (int32_t i = 0; i < 100000; ++i) {
    std::cout << "Hello World\n";           // faster
} // for i
```

- `std::endl` does not just print a newline, it also flushes the buffer. It is equivalent to:

```
std::cout << '\n' << std::flush;
```

- The first loop flushes on every single iteration. `'\n'` lets output accumulate and print in chunks.

When endl is preferred

- If output is buffered and the program crashes before a flush, the buffered information is **lost**.

```
281 std::cout << "Reached line 281\n";
...
311 raise(SIGSEGV); // segfault here
312 std::cout << "Reached line 312\n";
```

Console output:

```
Segmentation fault (core dumped)
```

- It looks like the program never reached line 281. With `std::endl` instead, the output is flushed immediately:

```
281 std::cout << "Reached line 281" << std::endl;
...
311 raise(SIGSEGV);
312 std::cout << "Reached line 312" << std::endl;
```

```
Reached line 281
Segmentation fault (core dumped)
```

- Now you know the program passed line 281 and crashed before line 312. This is also why `std::cerr` is unbuffered, and why error output should be sent to `std::cerr` instead of `std::cout`.

2.6 C and C++ Stream Synchronization TAKEAWAY ONLY

The takeaway: as long as you are not mixing C and C++ style I/O, putting `std::ios_base::sync_with_stdio(false);` on the first line of `main()` can significantly speed up I/O.

- `std::cin` and `std::cout` are C++ objects. C has three different standard streams: `stdin`, `stdout`, and `stderr`, and uses `printf()` for output and `scanf()` for input instead of the insertion and extraction operators. You do not need to work with C in this class.
- Since C and C++ I/O can be used together, the two must be **synchronized** so output appears in the right order. C++ ensures this by making each C++ stream share a buffer with its C counterpart (`std::cout` and `stdout` write to the same buffer).

```
int main() {
    printf("EECS"); // C stream
    std::cout << "281"; // C++ stream
} // main()
```

- "EECS" is sent to an output stream first, so it prints first. This is **stream synchronization**.
- Synchronization costs time, because extra work is needed to maintain correct ordering. Turn it off with:

```
std::ios_base::sync_with_stdio(false);
```

- This can speed up execution noticeably for programs that process a lot of input or output.
- **Side effect:** do not add this line if your program uses both C and C++ style I/O.

```
int main() {
    std::ios_base::sync_with_stdio(false);
    printf("EECS");
    std::cout << "281";
} // main()
```

- With synchronization off, the two streams have independent buffers. Depending on which flushes first the output could be "EECS281", "281EECS", or even interleaved like "E2E8C1S". Without synchronization, all bets are off.
- You will not use C-style I/O in this class, so turning synchronization off is fine for projects. Just remember the improved performance is a side effect, and do not blindly add the line to every program because it might speed things up.

2.7 Stringstreams

Marked as extra material in the notes. NOT REQUIRED

- A **stringstream** lets you treat a `std::string` as if it were a stream, so you can read from or write to it like any other stream. Include `<sstream>`.
- **std::istringstream** treats a string as an **input** stream. It inherits from `std::istream`, so it can be used anywhere a `std::istream` is expected (polymorphism). The `str()` member function sets the contents of the stream.

```
std::istringstream iss;
std::string numbers = "2 33 15 23";
iss.str(numbers);           // initializes iss with the contents of numbers
int32_t val;
for (int32_t i = 0; i < 4; ++i) {
    iss >> val;             // extracts a number from iss
    std::cout << val * 2 << ' ';
} // for i
// output: "4 66 30 46 "
```

- **std::ostringstream** treats a string as an **output** stream. It inherits from `std::ostream`, so it can be used anywhere a `std::ostream` is expected. Write with `<<`, then convert the contents to a string with `str()`.

```
std::ostringstream oss;
oss << "I really enjoy EECS 281";
oss << "!!!";
std::string s = oss.str();    // converts contents of oss to a string
std::cout << s << '\n';     // "I really enjoy EECS 281!!!"
```

2.8 File Streams

Marked as extra material in the notes. **NOT REQUIRED**

- Input redirection is not the only way to read and write files. **File streams** live in `<fstream>`.
 - `std::ifstream` (input file stream): reads from a file.
 - `std::ofstream` (output file stream): writes to a file.
 - `std::fstream`: can do both.

Important `std::ifstream` member functions:

Function	Behavior
<code>void open(const char* filename);</code>	Opens <code>filename</code> and associates it with the input file stream
<code>bool is_open();</code>	Checks whether the stream is associated with a successfully opened file
<code>void close();</code>	Closes the file and disassociates it from the stream
<code>operator>></code>	Extracts input from the stream (inherited from <code>std::istream</code>)

Once associated with a file, an `ifstream` is used like any other input stream. Good practice is to check `is_open()` before extracting.

```
// create ifstream object and associate it with input.txt
std::ifstream fin;
fin.open("input.txt");

// check if the file was successfully opened
if (!fin.is_open()) {
    std::cerr << "Failed to open file." << std::endl;
    exit(1);
} // if

// read from the file
std::string word;
while (fin >> word) {
    // do stuff here
} // while

fin.close(); // close the file when done
```

`std::ofstream` has the same `open/is_open/close` functions, plus the insertion operator:

Function	Behavior
<code>void open(const char* filename);</code>	Opens <code>filename</code> and associates it with the output file stream
<code>bool is_open();</code>	Checks whether the stream is associated with a successfully opened file
<code>void close();</code>	Closes the file and disassociates it from the stream
<code>operator<<</code>	Inserts output into the stream (inherited from <code>ostream</code>)

```
std::ofstream fout;
fout.open("output.txt");

if (!fout.is_open()) {
    std::cerr << "Failed to open file." << std::endl;
    exit(1);
} // if

fout << "This data is written to the file." << std::endl;

fout.close();
```

- `std::fstream` supports both insertion and extraction, but prefer `std::ifstream` and `std::ofstream` unless you truly need read and write access to one file through a single object (uncommon). Separate types make the intent clear and reduce human error.

2.9 Reading Input Using Polymorphism

Marked as extra material in the notes. NOT REQUIRED

- Recap of redirection:
 - Reading input from a file is like temporarily disconnecting the keyboard and taking all input from a file (**input redirection**, <).
 - Writing output to a file is like temporarily disconnecting the screen and sending everything to a file (**output redirection**, >).
 - The output file does not need to exist beforehand, it is created as needed. If it already exists, its contents are **replaced**.
 - Redirection can be applied to input, output, both, or neither, and also to `std::cerr` using `2>`.
- A program often reads from a file named on the command line, but falls back to standard input if none is given. These two invocations should behave identically:

```
./program < input.txt      ./program input.txt
```

- The first uses input redirection; the second names the file directly as a command line argument and must be read with a file stream.
- Polymorphism makes supporting both easy: `std::ifstream` inherits from `std::istream`, so a single `std::istream` object can refer to either an input file stream or `std::cin`.

```
std::ifstream fin;
std::string filename;    // use argc and argv to get the file name from the
if (argc > 1) { ... }    // command line if specified (see chapter 3)

if (!filename.empty()) {
    fin.open(filename);
    if (!fin.is_open()) {
        std::cerr << "Unable to open input file: " << filename << std::endl;
        exit(1);
    } // if
} // if

std::istream& in = fin.is_open() ? fin : std::cin;

// Read input by extracting from 'in' for the rest of the program (in >> ...)
// If a file was specified, 'in == fin' (extracts from the file stream)
// If no file was specified, 'in == cin' (extracts from standard input)

fin.close();
```

- The ternary line is where polymorphism comes in. `in` is a **reference** to whichever stream is correct, so the rest of the program reads from `in` without caring whether the data came from a file stream or standard input.